BundleFuse

The pipeline implemented in code

A step-by-step mathematical specification

Derived directly from the current working source

3 November 2026

Abstract

This document follows the executable pipeline: resolve configuration, construct data folds, build fixed artifacts, compute and standardize candidate signals, select fusion settings on validation queries, save a fitted checkpoint, predict, and evaluate. It describes both the POG/Spotify path and the distinct retail/commerce path, as well as the optional extended profile. Equations specify what the functions compute; interpretation is separated from implementation facts. Earlier documentation, paper descriptions, and historical result tables are not sources for this account. No accuracy or novelty claim is inferred from the formulas.

Snapshot. The branch is `reorganized`. Its base commit is

`162dd8ce8846b91bca3b508ede45fc6ebeaab576`

The working tree contains changes, so that commit alone does not identify this implementation. The companion `pipeline_theory_sources.json` records SHA-256 hashes for the 30 inspected runtime, configuration, entry-point, and timing-script files. Their combined source-set digest is

`d9fe136d0d7a084d3c2e99510e5ea4acde35785b4515bf3a5e6018b8366b8033`

The source map in Appendix E connects the derivations to functions.

Contents

Notation and the execution contract **4**

1 Step 1: resolve configuration and dispatch **5**

2 Step 2: construct the data and query contract **6**

2.1 Supplied POG/Spotify folds 6

2.2 Random resplit and reveal masking 6

2.3 Default Dunnhumby preprocessing 7

2.4 Default commerce preprocessing 7

2.5 Text features and information available before splitting 7

3 Step 3: build the common training graph **8**

3.1 Co-occurrence, positive PMI and collaborative SVD 8

3.2 The zero-diagonal propagation operator 9

3.3 Two different pairwise objects 9

3.4 Pure incidence surplus 9

4 Step 4: build dense content and ontology artifacts **9**

4.1 Input normalization, category labels and style residuals 9

4.2 Dense bridge and optional content projection 10

4.3 Category complementarity and saturation 10

4.4 Soft ontology for the general non-music path 11

5 Step 5: build the research basket context artifacts **11**

5.1 Retail household context and bridge 11

5.2 Commerce user/session graphs and external pretraining graph 12

6 Step 6: compute raw candidate signals **12**

6.1 Shared collaborative, diffusion and content signals 12

6.2 Taxonomy, homophily and household scores 13

6.3 Concordance: support from pairs of revealed seeds 13

7 Step 7: standardize signals and fix their ordered menu **14**

8 Step 8: select the frozen fusion on validation queries **15**

8.1 Define the ranking rule and validation objective first 15

8.2 Define observed evidence, prior and the final score 16

8.3 Why this is called an empirical-Bayes router 16

8.4 Stage A: coordinate search and backward pruning . . . 16
8.5 Stage B: choose prior keys and higher-order settings . . . 17
8.6 Stage C: freeze prior scale and estimate exposure moments . . . 17
8.7 Stage D: choose the shrinkage setting . . . 18
8.8 How validation memory and replay work . . . 19

9 Step 9: save fitted state and restore it exactly 19

10 Step 10: execute a new prediction 20
10.1 Validate the query and choose scoring seeds . . . 20
10.2 Determine dependencies of the frozen fusion . . . 20
10.3 Compute, standardize, fuse and rank . . . 20

11 Step 11: run evaluation and bind it to the fitted data 21
11.1 Recorded fingerprints and actual acceptance checks . . . 21
11.2 Macro test metrics and seed experiments . . . 21

12 A small numerical walk-through 22

A Extended-profile behavior and additional signals 23
A.1 Preprocessing differences . . . 23
A.2 Auxiliary co-occurrence and group signals . . . 23
A.3 Optional personalization . . . 23
A.4 Extended menu order . . . 24

B Baseline paths implemented beside BundleFuse 24

C Runtime cost and the timing entry point 24

D Active pipeline versus helper methods 25

E Source map and reproduction 25

Notation and the execution contract

All core symbols are defined here before use. Additional symbols are introduced locally, immediately before their equations. Bold lower-case letters are column vectors; matrices are upper-case. A matrix row stores an item's features, and a score-matrix column stores one query's scores over the entire catalogue.

Symbol	Definition
n, $\mathcal{I}$	Number of catalogue items and the internal ID set $\mathcal{I} = \{0, \dots, n-1\}$.
m, b_e	Number of training bundles and training bundle e, $1 \le e \le m$. Different training rows may contain the same item set.
S_j, G_j	Original revealed seeds and held-out ground-truth items for query j; $S_j \cap G_j = \varnothing$.
$S_j^\star$	Seeds actually used for scoring after the optional seed cap. All original S_j are excluded from recommendations.
v, q	Number of validation queries and number of queries in a scoring batch.
$\boldsymbol{x}_j$, X	Indicator $x_{ij} = \mathbf{1}[i \in S_j^\star]$ and batch matrix $X = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_q] \in \mathbb{R}^{n \times q}$.
B, B_0	Training bundle–item incidence and its binary incidence, both $m \times n$. In ordinary `BundleFuse.fit`, both are binary and equal.
f_i, d_e	Training exposure of item i and size of bundle e.
ℓ_i, a_i	Item category/artist label and, when available, album label.
F	A generic dense item-feature matrix; F_i is row i. A subscript identifies a particular feature family.
C, M, U	Zero-diagonal item co-occurrence, its positive-PMI matrix, and row-normalized collaborative SVD features.
r_{ij}^t, z_{ij}^t	Raw and catalogue-standardized score of signal key t for item i, query j.
$\mathcal{K}$, $\mathcal{P}$	Ordered selectable signal menu and selected prior-key list.
w_t	Nonnegative, validation-selected weight of signal key t; zero keys are removed after pruning.
$\alpha_d, \gamma_v, \beta$	Item-exposure exponent, training-bundle volume exponent, and incidence-surplus coefficient. Defaults: $0.5, 0.5, 1.5$.
α_b	Dense content-bridge mixing coefficient; default 0.5.
α_h, γ_R	Validation-selected incidence-mixture and concordance coefficients. These differ from α_d, α_b and the concordance kernel parameter.
Y_{ij}, P_{ij}	Fused observed score and unscaled prior score.
s_0, κ	Validation-frozen prior scale and exposure shrinkage ratio.
$\widehat{\theta}_{ij}$	Final fused score before original revealed items are excluded.
K	Requested recommendation or metric cutoff.

Basic operations. Define $[x]_+ = \max(0, x)$, elementwise product $\odot$, $\operatorname{diag}(\boldsymbol{u})$ as a diagonal matrix, and $\operatorname{off}(A)$ as A with diagonal set to zero. For dense rows define

$$\mathcal{N}(F)_i = \frac{F_i}{\|F_i\|_2 + 10^{-8}}. \tag{N.1}$$

Sparse text rows use scikit-learn's exact L2 normalization, which leaves zero rows zero; it is not the epsilon-normalization in (N.1). Most final signal arrays are float32. The formulas below describe real-valued arithmetic; where operation order or precision matters, the actual code order is stated explicitly.

What fitting means. There is no gradient-trained neural network in this path. Fitting constructs empirical graph statistics, SVD features and sometimes k-means clusters, then searches validation-selected weights and routing settings. These are data-fitted artifacts and selected parameters; calling the algorithm "parameter-free" would obscure that distinction. Test ground truth is not passed into `BundleFuse.fit`.

End-to-end order.

1. Resolve a dataset definition, preprocessing profile, split protocol, and model configuration.
2. Load training bundles, validation queries, test queries, catalogue features, and context metadata.
3. Build training-incidence operators and the appropriate content/context artifacts.
4. Compute and standardize validation signal columns over all catalogue items.
5. Select weights, higher-order mixture, prior scale, and shrinkage on validation only.
6. Retain fitted state; optionally evaluate and save it through the CLI.
7. For a new query, compute only the dependencies of the frozen fusion, then rank.
8. Evaluate with the same ranking rule and the checkpoint's reconstructed data protocol.

The fit CLI evaluates before saving by default; saving is skipped with `--no-save`, and evaluation is skipped with `--no-eval`. The conceptual distinction between fitting and inference is unchanged by that ordering.

1 Step 1: resolve configuration and dispatch

Code: `bundlefuse/datasets/__init__.py; bundlefuse/config.py; evaluator.py`

The registry maps eight dataset names into three loader families. Shared methods implement `datasets.load(name)`, `datasets.get(name).config(n_items)`, and `datasets.build(name)`. The last returns an *unfitted* model and loaded dataset. The standard fit call is

```
model, ds = datasets.build("pog_dense")
model.fit(ds.train, ds.val_pairs, **ds.fit_kwargs())
```

`fit_kwargs()` forwards features, labels, optional context inputs, validation users, preprocessing-specific data, and a copied provenance specification. Dataset loading by itself does not fit or save a model.

Datasets	Loader family	Default protocol	Ordinary query limits
POG, POG Dense, Spotify, Spotify Sparse	BundleMLLM	Supplied files	All validation/test queries
Dunnhumby	Retail	Random resplit	800 validation, 2,000 test
Electronic, Food, Clothing	BundleRec commerce	Random resplit	800 validation, 2,000 test

Table 2: Registry and fit-CLI defaults. Explicit zero or explicit Python `None` limits mean full folds. `validate.py` passes `None` by default and therefore uses full folds.

There are *two independent profile settings*. The `data_profile` chooses preprocessing; the model's `signal_profile` chooses the scorer/selection recipe. Both support `research` and `extended`. All

eight current dataset JSON files set `signal_profile="research"`; default dataset loading also chooses `data_profile="research"`. In contrast, constructing `BundleFuseConfig` directly defaults its signal profile to `extended`. A caller that disables JSON configuration does not automatically recover the dataset JSON's research profile.

Configuration precedence is: dataset JSON (or fallback when no JSON is used), then non-null caller overrides, then explicitly supplied CLI overrides. The catalogue size comes from the loaded dataset. Current JSON files set collaborative SVD dimension 128, $\alpha_d = \gamma_v = 0.5$, $\beta = 1.5$, $\alpha_b = 0.5$, `compressed=true`, and `content_rank=null`. Thus the current default does *not* apply an additional content projection, despite compressed checkpoint storage. Other dataclass defaults include four coordinate-search rounds, 32-query validation blocks, 400 soft clusters, and a zero seed cap (use all seeds).

After the common graph is built, the special basket path is taken exactly when `signal_profile == 'research'` and the preprocessing payload's kind is `retail` or `bundlerec`. Otherwise fitting uses the general path. A research signal setting without the basket payload therefore does not activate basket scoring. Research baskets require their full family set; family ablations use the extended path. They also reject a non-null content projection or $\alpha_b \neq 0.5$ rather than silently ignoring those overrides.

2 Step 2: construct the data and query contract

Code:
`bundlefuse/datasets/base.py; bundlefuse/datasets/baskets.py; bundlefuse/splits.py`

2.1 Supplied POG/Spotify folds

`ProvidedSplits` reads training rows and paired input/ground-truth files for validation and test. It checks catalogue size, item bounds, unique row IDs, nonempty rows, no duplicate items inside a row, matching input/target query IDs, disjoint fold IDs, and no seed/target overlap within a query. Query pairs are emitted in ascending original bundle-ID order. Distinct training IDs with identical item contents are retained. Supplied mode neither regenerates masks nor deduplicates bundles. Non-default resplit seeds or mask fractions are rejected in supplied mode.

For non-music data, available title, image and description arrays are loaded separately. For music, the available content tensor is loaded under the audio key. Each modality is row-normalized by (N.1); their concatenation is also supplied as clustering input. Fashion labels come from `cate`/`cate_id`; music labels are artist indices with separate album indices. Music is recognized by dataset name or artist metadata.

2.2 Random resplit and reveal masking

Let M_b denote the number of retained complete bundles and let r be the requested split seed. Resplitting shuffles their index/order list with Python's `random.Random(r)`; training uses the first $\lfloor 0.7M_b \rfloor$ bundles, validation the next $\lfloor 0.1M_b \rfloor$, and test the remainder. For POG/Spotify resplit mode, complete bundles are first reconstructed from supplied inputs plus targets, deduplicated by sorted item set, and restricted to at least two distinct items.

Let $\phi \in (0, 1)$ be the hidden fraction, default 0.5, and let $\widetilde{b}$ be a shuffled copy of a held-out complete bundle. Before forming each query define the reveal count

$$h(b) = \max\{1, \mathrm{round}((1 - \phi)|b|)\}. \tag{2.1}$$

The first $h(b)$ shuffled items form S, and the rest form G. If the latter is empty, the last item is hidden and the others are revealed. Python rounding, including its half-to-even rule, is used. A single RNG is reused across all bundles in each masking call; validation uses seed $r + 1$, test $r + 2$. Positive query caps retain a prefix after masking; zero/`None` retain the full fold. A query cap is not a new random sample.

2.3 Default Dunnhumby preprocessing

1. Group transaction rows by basket ID into unique product sets, retaining household and day metadata.
2. Keep baskets of 3–20 products; shuffle once with seed zero and cap at 30,000 baskets.
3. Count product appearances in that capped pool, retain products occurring at least twice, remove other products, and discard resulting baskets shorter than three items.
4. Map retained products, sorted by original ID, to contiguous internal IDs. Globally deduplicate identical mapped baskets, keeping the first household/day association.
5. Read product text from department, commodity, subcommodity and brand. Subcommodity gives a category; a product with missing metadata receives its own missing category.
6. Fit catalogue TF–IDF and SVD content features as described below, then resplit and mask. Household counts used by the model are built from the training fold only.

The day column is read, but this loader uses a random split, not a chronological split.

2.4 Default commerce preprocessing

1. Read bundle–item rows, build the catalogue from all listed items, deduplicate items within bundles, retain bundles of at least two items, and globally deduplicate equal item sets.
2. Retain the first original bundle key and load its user and session associations.
3. Read the external user–item pretraining file and restrict its items to this catalogue. Its weighted co-occurrence graph is defined in Section 5.
4. Parse the first bracketed category path. The entire parsed path defines the leaf label; its first up-to-two entries define a second-level label. Missing labels are unique per item.
5. Construct text by concatenating the comma-joined title fields and parsed category path. Build TF–IDF/SVD features, then shuffle, resplit and mask the retained bundles.

User-bundle timestamps are read but not used for a temporal restriction. User/session co-occurrence from bundle interactions uses training bundles only; the external pretraining file is loaded as supplied, with no timestamp filtering in this code.

2.5 Text features and information available before splitting

Let c_{it} be the count of retained token/ngram t in item document i, and let df_t be the number of catalogue documents containing t. The configured vectorizer uses English stop words, unigrams and bigrams, and sublinear term frequency. With the installed library's smoothed-IDF defaults,

the pre-normalized text weight is

$$t_{it} = \begin{cases} (1 + \log c_{it}) \left(1 + \log \dfrac{n+1}{\mathrm{df}_t + 1}\right), & c_{it} > 0, \\ 0, & c_{it} = 0. \end{cases} \tag{2.2}$$

Let $T \in \mathbb{R}^{n \times d_t}$ be the resulting sparse L2-normalized text matrix, where d_t is its retained vocabulary size. The vocabulary cap is 8,000 for retail and 12,000 for default commerce. Define $r_t = \min(128, d_t - 1)$, and let V_t denote the right singular vectors returned by `TruncatedSVD(r_t, random_state=0)`. Dense basket content is $F_c = \mathcal{N}(TV_t)$.

This preprocessing is catalogue-transductive: text vocabulary/SVD use all retained item metadata before the bundle split. Retail's frequency filter also examines the pre-split basket pool. This differs from fitting graph co-occurrence on training bundles. The code therefore supports a known catalogue, not a claim that every preprocessing decision is estimated from the training fold alone. Predictions use contiguous internal IDs; the loaders do not export a dedicated original-ID mapping inside the fitted model.

3 Step 3: build the common training graph

Code: `bundlefuse/operators.py: SimplicialGraphOperator`

In ordinary model fitting define $B_{ei} = \mathbf{1}[i \in b_e]$ and $B_0 = B$; repeated items inside a training row are removed by the graph constructor. The helper also accepts bundle weights, but `BundleFuse.fit` does not pass them. The equations in this section apply to the binary incidence used by that public fit path. Define

$$f_i = \sum_e B_{ei}, \qquad d_e = \sum_i (B_0)_{ei}, \tag{3.1}$$

$$D = \mathrm{diag}((f_i + 5)^{-\alpha_d}), \qquad V = \mathrm{diag}((d_e + 1)^{-\gamma_v}), \qquad E = \mathrm{diag}((d_e + 1)^{-1}). \tag{3.2}$$

f_i counts appearances in retained *training rows*, so retained duplicate rows affect exposure. The additive constants 5 and 1 are implementation choices, not symbolic limits.

3.1 Co-occurrence, positive PMI and collaborative SVD

Define $g_i = \sum_j C_{ij} + 10^{-9}$ and $z_C = \sum_{i,j} C_{ij}$ after computing

$$C = \mathrm{off}(B^\top B). \tag{3.3}$$

For stored nonzero co-occurrences define

$$M_{ij} = \left[\log\left(\frac{C_{ij} z_C}{g_i g_j} + 10^{-12}\right)\right]_+; \qquad M_{ij} = 0 \text{ when } C_{ij} = 0. \tag{3.4}$$

The denominator uses co-occurrence row masses g_i, not bundle exposure f_i. Let $r_c = \min(\texttt{svd_dim}, \max(1, n-2))$ and V_c be the truncated right singular basis of M. The stored collaborative feature matrix is

$$U = \mathcal{N}(MV_c). \tag{3.5}$$

The SVD uses random seed zero; if M has no nonzeros, U is a zero matrix of the requested clamped width. The feature is the transformed matrix (left singular vectors times singular values), followed by row normalization, not the unscaled left singular vectors alone.

3.2 The zero-diagonal propagation operator

Define the symmetric propagation operator $L \in \mathbb{R}^{n\times n}$ by

$$L = DB^\top EBD - \operatorname{diag}\left(D_{ii}^2 \sum_e B_{ei}E_{ee}\right). \tag{3.6}$$

Since B is binary, the subtracted vector is the diagonal of the preceding product. `propagate(A)` computes LA by multiplying in the order $B(DA)$, then $B^\top E[B(DA)]$, then left-scaling by D and subtracting the diagonal correction. It accepts a score vector or an n-row dense feature/score matrix and returns float32. It does not use row-stochastic normalization.

3.3 Two different pairwise objects

For a query indicator $\boldsymbol{x}$ define its training-bundle overlap vector $\boldsymbol{o} = VBD\boldsymbol{x}$. The ordinary `pair` score is

$$\boldsymbol{p} = B^\top \boldsymbol{o} = B^\top VBD\boldsymbol{x}. \tag{3.7}$$

Separately, the graph stores

$$W = \operatorname{off}(B^\top VBD), \tag{3.8}$$

under `W_pair`. W is used for concordance; (3.7) is used for the `pair` signal. They differ on the diagonal. Although revealed candidates are excluded at the final ranking step, their scores participate in standardization and diffusion. Thus replacing (3.7) by $W\boldsymbol{x}$ would change the implemented pipeline.

3.4 Pure incidence surplus

Define overlap counts $\boldsymbol{k} = B_0\boldsymbol{x}$ and let $\boldsymbol{\delta}$ denote the incidence-surplus score vector. The code computes

$$\boldsymbol{\delta} = B^\top\left[\boldsymbol{o} \odot \beta\left[\boldsymbol{k} - \mathbf{1}\right]_+\right]. \tag{3.9}$$

For candidate i, this adds evidence from training bundles that contain i and at least two revealed seeds. It is exactly zero for a single-seed query or whenever no training bundle overlaps two scoring seeds. This is excess over a linear overlap score; the code never requires all revealed seeds to occur in the same training bundle.

4 Step 4: build dense content and ontology artifacts

Code: `bundlefuse/model.py: BundleFuse.fit; bundlefuse/operators.py: DualSpectrumOntology, BilateralContentBridge`

This section describes the general POG/Spotify path. Research basket content/context artifacts are specified separately in Section 5.

4.1 Input normalization, category labels and style residuals

Let $F^{(1)}, \ldots, F^{(b)}$ be the available content modalities, with b denoting the number of modalities. The loader normalizes each modality. A research full-ranking preprocessing payload marks them as already normalized; otherwise the model normalizes them again. Their raw content feature is

$$F_{\text{content}} = \mathcal{N}([F^{(1)}|\cdots|F^{(b)}]). \tag{4.1}$$

If explicit category labels are absent, the model obtains them by k-means on supplied clustering features, concatenated modalities, or collaborative features, in that priority. The effective cluster count is $k_s = \max(2, \min(\texttt{n_soft_clusters}, n-1))$. Every k-means call uses `MiniBatchKMeans`, random seed zero, three initializations, and batch size 4,096.

For each modality, define a style-group label c_i. For fashion, $c_i = \ell_i$; for music, c_i is a k-means cluster of audio/content features, not the artist label. For each group c define its catalogue centroid $\overline{F}_c$, then its normalized residual feature

$$\overline{F}_c = \frac{1}{|\{i : c_i = c\}|} \sum_{i:c_i=c} F_i, \qquad F_{\text{style},i} = \frac{F_i - \overline{F}_{c_i}}{\|F_i - \overline{F}_{c_i}\|_2 + 10^{-8}}. \tag{4.2}$$

The keys are `style`, `imgstyle`, `descstyle`, and `audiostyle` for title/content, image, description, and audio respectively. These are deterministic centroid residuals after clustering/label construction, not outputs of a neural encoder.

4.2 Dense bridge and optional content projection

Let F_s be the normalized concatenation of available style residuals. In the research music path, F_s is the audio style residual itself, with no extra normalization. Define the propagated and bridged features

$$F_p = \mathcal{N}(LF_s), \qquad F_{\text{bridge}} = \mathcal{N}(F_s + \alpha_b F_p). \tag{4.3}$$

This mixes item content with graph-neighbor content once. It is built during fitting and reused at prediction time.

Only when both `compressed=true` and `content_rank=r` is non-null does the model apply an additional rank-r truncated-SVD transform to any content/style/bridge feature whose input width exceeds r. Each eligible feature is projected separately, using seed zero. Projected rows are *not* normalized again. Collaborative U is excluded from this projection. The concordance descriptor is retained separately before projection, so projecting the scoring feature table does not necessarily project concordance features. With the current JSON defaults, this entire additional projection step is skipped.

4.3 Category complementarity and saturation

Let c index the n_c distinct category labels, and define the training category-count matrix $A \in \mathbb{R}^{m \times n_c}$ by $A_{ec} = |\{i \in b_e : \ell_i = c\}|$. Define category masses $a_c = \sum_e A_{ec} + 10^{-9}$, total $a_+ = \sum_c a_c$, and $H = \text{off}(A^\top A)$. For nonzero H_{cd} define

$$Q_{cd} = \left[\log\left(\frac{H_{cd} a_+}{a_c a_d} + 10^{-12}\right)\right]_+; \quad Q_{cd} = 0 \text{ otherwise.} \tag{4.4}$$

The marginal in this category formula is a count of category appearances, not the row sum of H. Different items with the same label contribute repeated counts to A.

For each category c, define u_c as the number of training bundles containing that category at least once and v_c as the number containing it at least twice. Define the smoothed duplication rate $\rho_c = (v_c + 0.5)/(u_c + 1)$. The non-music saturation score is

$$r_i^{\text{slot}}(S^\star) = -\sum_{s \in S^\star} \mathbf{1}[\ell_i = \ell_s]\, \mathbf{1}[\rho_{\ell_s} < 0.15]\, (1 - \rho_{\ell_s}). \tag{4.5}$$

It penalizes a candidate sharing a rarely duplicated category with each matching seed. The general fashion category signal adds one half of this score. General music and the research basket category signals do not add this term.

4.4 Soft ontology for the general non-music path

Let F_o be the concatenated modality input used by the ontology. The research fashion path row-normalizes this concatenation; the extended path uses the concatenation directly. Let $\boldsymbol{\mu}_c$ be k-means centre c, normalized to unit length with epsilon 10^{-8}, for $1 \le c \le k_s$. Let τ_s be `soft_tau`, default 0.05. The soft membership matrix $J \in \mathbb{R}^{n \times k_s}$ is

$$t_{ic} = F_{o,i}\boldsymbol{\mu}_c^\top/\tau_s, \qquad J_{ic} = \frac{\exp(t_{ic} - \max_d t_{id})}{\sum_d \exp(t_{id} - \max_d t_{id}) + 10^{-9}}. \tag{4.6}$$

Define each training bundle's soft sum $\boldsymbol{b}_e^s = \sum_{i \in b_e} J_i$ and the matrix B_s with those rows. The off-diagonal nonnegative soft co-occurrence is

$$H_s = \left[\operatorname{off}\left(B_s^\top B_s - \sum_e \sum_{i \in b_e} J_i^\top J_i\right)\right]_+. \tag{4.7}$$

The subtracted term removes each item's self-outer-product inside a training bundle. Define catalogue soft masses $a_c^s = \sum_i J_{ic}$ and $a_+^s = \sum_c a_c^s$. The stored soft compatibility matrix is computed over all entries as

$$(Q_s)_{cd} = \left[\log\left(\frac{(H_s)_{cd}a_+^s}{a_c^s a_d^s + 10^{-9}} + 10^{-12}\right)\right]_+. \tag{4.8}$$

The model stores J and JQ_s. The soft signal later matches a candidate's soft compatibility row against the *sum* of seed membership rows. This is separate from the category-count PMI in (4.4). If the ontology has no clustering features, these soft artifacts are absent and `softcat` is unavailable.

5 Step 5: build the research basket context artifacts

Code: `bundlefuse/datasets/baskets.py; bundlefuse/basket_signals.py: BasketSignals`

The research basket path returns before the general ontology/style construction. Its content F_c is the catalogue text-SVD matrix from Section 2.5. Its category PMI uses the count construction of (4.4), without slot saturation or soft labels.

5.1 Retail household context and bridge

Let Z_h be the household-by-item training count matrix; $(Z_h)_{ui}$ counts appearances of item i in training baskets belonging to household u. Let $d_u^h = \sum_i (Z_h)_{ui} + 1$. Define

$$\widetilde{Z}_h = \operatorname{diag}((d_u^h)^{-1/2})Z_h, \qquad C_h = \operatorname{off}(\widetilde{Z}_h^\top \widetilde{Z}_h). \tag{5.1}$$

The retail household signal will use $C_h D\boldsymbol{x}$, including the common seed-exposure weight D. Its dense bridge is

$$F_h = \mathcal{N}\big(F_c + 0.5\mathcal{N}(LF_c)\big). \tag{5.2}$$

Household context is shared evidence across the training fold. It is not a per-query household-personalization term in this research basket path; supplied query users do not change `BasketSignals.score`.

5.2 Commerce user/session graphs and external pretraining graph

Let Z_u and Z_s be user-by-item and session-by-item count matrices built only from training bundles and their recorded associations. Missing negative associations are skipped. Define $C_u = \mathrm{off}(Z_u^\top Z_u)$ and $C_s = \mathrm{off}(Z_s^\top Z_s)$; these two contexts are not degree-normalized here.

Let Z_p be the external pretraining user-by-item count matrix after catalogue restriction, and define $d_u^p = \max(\sum_i (Z_p)_{ui}, 1)$. Define its degree-normalized co-occurrence and the commerce diffusion operator L_c as

$$C_p = \mathrm{off}\left(Z_p^\top \mathrm{diag}((d_u^p)^{-1}) Z_p\right), \tag{5.3}$$

$$D_c = \mathrm{diag}((f_i + 1)^{-1/2}), \tag{5.4}$$

$$L_c = D_c(C + 0.5C_s + 0.5C_u + 0.2C_p)D_c. \tag{5.5}$$

L_c replaces L for commerce score diffusion. It does not replace L inside the common graph constructor or the separately stored concordance matrix W.

Using sparse text T and dense text-SVD content F_c, define

$$T_b = \mathrm{L2rows}(T + 1.5L_cT + 0.5L_c(L_cT)), \tag{5.6}$$

$$F_b = \mathcal{N}(F_c + 1.5L_cF_c + 0.5L_c(L_cF_c)). \tag{5.7}$$

T_b is the selectable sparse bridge; F_b is a dense descriptor for concordance. The two are different artifacts, and the dense one is not an additional selectable `content` signal in the default commerce menu.

For commerce taxonomy, build three versions of (4.4): leaf-category Q_ℓ^{off} with diagonal removed, leaf-category Q_ℓ^{keep} with diagonal retained, and second-level Q_2^{keep} with diagonal retained. In each version the appropriate label-count matrix supplies its own marginals. The last two support the commerce homophily signal.

6 Step 6: compute raw candidate signals

All formulas here use scoring seeds $S^\star$ and indicator $\boldsymbol{x}$. They produce one score for *every* catalogue item, including seeds; seed exclusion occurs only when ranking. Batch implementations perform these same operations for columns of X.

6.1 Shared collaborative, diffusion and content signals

Define $\boldsymbol{p}$ and $\boldsymbol{\delta}$ by (3.7) and (3.9). In commerce, first replace the pair score by

$$\boldsymbol{p}_c = \boldsymbol{p} + 0.35C_u\boldsymbol{x} + 0.35C_s\boldsymbol{x}. \tag{6.1}$$

The family-specific raw score definitions are then:

Code key	General / research retail	Research commerce
`pair`	$\boldsymbol{p}$	$\boldsymbol{p}_c$
`simp`	$\boldsymbol{p} + \boldsymbol{\delta}$	$\boldsymbol{p}_c + \boldsymbol{\delta}$
`diff2_pair`	$L\boldsymbol{p}$	$L_c\boldsymbol{p}_c$
`diff2`	$L\boldsymbol{p} + L\boldsymbol{\delta}$	$L_c(\boldsymbol{p}_c + \boldsymbol{\delta})$
`cooc`	$C\boldsymbol{x}$	$C\boldsymbol{x}$
`ppmi`	$M\boldsymbol{x}$	$M\boldsymbol{x}$
`pop`	$\log(1 + f_i)$ for candidate i	Not in the selectable menu
`csvd`	$U_i\big(\lvert S^\star\rvert^{-1} \sum_{s\in S^\star} U_s\big)^\top$	Same

The code evaluates $L\boldsymbol{p}$ and $L\boldsymbol{\delta}$ separately before adding for general/retail `diff2`. Algebraically it equals $L(\boldsymbol{p}+\boldsymbol{\delta})$, but the prescribed float32 operation order is retained. Commerce uses the latter summed-input form with L_c.

For any selectable dense feature F_t, the raw content score is

$$r_i^t(S^\star) = F_{t,i}\left(\frac{1}{|S^\star|}\sum_{s\in S^\star} F_{t,s}\right)^\top. \tag{6.2}$$

This applies to raw content, styles, dense bridges, and collaborative SVD features when they are in the selected menu. Research retail uses F_c for `content` and F_h for `bridge`. Research commerce instead scores

$$\boldsymbol{r}^{\text{tfidf}} = T(T^\top \boldsymbol{x}), \quad \boldsymbol{r}^{\text{bridge}} = T_b(T_b^\top \boldsymbol{x}), \quad \boldsymbol{r}^{\text{pretrain}} = C_p\boldsymbol{x}. \tag{6.3}$$

Sparse text uses a *sum* of seed columns; dense (6.2) uses a mean. Retail also computes TF–IDF in its full reference scorer, but that key is not selectable in its default menu. Placeholder `contrast/img` values in the basket full scorer are `None`, not trained neural components.

6.2 Taxonomy, homophily and household scores

For the relevant category matrix Q, the basic category score is

$$r_i^{\text{cat}}(S^\star) = \sum_{s\in S^\star} Q_{\ell_s,\ell_i}. \tag{6.4}$$

General non-music scoring adds $0.5r_i^{\text{slot}}$ from (4.5). General music uses artist labels without that penalty. Commerce uses Q_ℓ^{off} without the penalty. Its homophily score, using second-level label h_i, is

$$r_i^{\text{homo}}(S^\star) = \sum_{s\in S^\star}\left[(Q_\ell^{\text{keep}})_{\ell_s,\ell_i} + 0.5(Q_2^{\text{keep}})_{h_s,h_i}\right]. \tag{6.5}$$

For soft ontology, the score is

$$\boldsymbol{r}^{\text{softcat}} = (JQ_s)\left(\sum_{s\in S^\star} J_s\right)^\top. \tag{6.6}$$

For general music, let H_a and H_b be the item-by-artist and item-by-album binary membership matrices. Let their column sizes be n_c^a and n_c^b. Define $\widetilde{H}_a = H_a\,\mathrm{diag}((n_c^a+5)^{-1/2})$ and $\widetilde{H}_b = H_b\,\mathrm{diag}((n_c^b+2)^{-1/2})$. Then

$$\boldsymbol{r}^{\text{homo}} = \widetilde{H}_a\widetilde{H}_a^\top\boldsymbol{x} + 2\widetilde{H}_b\widetilde{H}_b^\top\boldsymbol{x}. \tag{6.7}$$

The album term is zero if album labels are absent. Research retail's separate household signal is $\boldsymbol{r}^{\text{hh}} = C_hD\boldsymbol{x}$, code key `hh_cooc`.

6.3 Concordance: support from pairs of revealed seeds

The concordance routine uses W in (3.8), not the `pair` score matrix. Let F_R be the descriptor used only for this routine, η the seed-kernel parameter, λ_d the dense-support coefficient, and $\tau_R = 0.05$ its denominator stabilizer. Before defining the kernel, define candidate-to-seed support and total support by

$$q_{is} = W_{is} + \lambda_d[F_{R,i}F_{R,s}^\top]_+, \qquad A_i = \sum_{s\in S^\star} q_{is}. \tag{6.8}$$

If no descriptor is available, omit the dense term. For seeds s, t, define similarity $G_{st} = 0.5\mathbf{1}[\ell_s = \ell_t] + 0.5[F_{R,s}F_{R,t}^\top]_+$ when both labels and features are available. With only one source use that source alone; with neither, the routine starts with the identity. Its diagonal is set to one. Define the off-diagonal seed kernel K^S by

$$K^S_{st} = \begin{cases} 0, & s = t, \\ [1 + \eta G_{st}]_+, & s \neq t,\ \eta < 0, \\ 1 + \eta G_{st}, & s \neq t,\ \eta \geq 0. \end{cases} \tag{6.9}$$

The raw concordance score is

$$r_i^{\mathrm{rao}}(S^\star) = \frac{\sum_{s \in S^\star} \sum_{t \in S^\star} q_{is} K^S_{st} q_{it}}{A_i + \tau_R}. \tag{6.10}$$

The double sum includes both ordered directions of each distinct seed pair; it is not halved. It vanishes for fewer than two scoring seeds. For $A_i > 0$, setting $p_{is} = q_{is}/A_i$ rewrites (6.10) as $A_i^2(A_i + \tau_R)^{-1} \sum_{s \neq t} K^S_{st} p_{is} p_{it}$. This equivalent form explains agreement weighting; the implementation uses the quadratic matrix expression directly, without dividing by A_i.

Path	Descriptor F_R	η	λ_d
General fashion	Title/content style residual	-1	0.15
General music	Audio style residual	$+1$	0.15
Research retail	Dense retail bridge F_h	-1	0.15
Research commerce	Dense commerce bridge F_b	-1	0.25

The general full scorer creates this signal only when its descriptor exists. The selected scorer requests it only when the frozen fusion needs it. η inside the kernel is *not* the validation-selected multiplier γ_R later applied to the entire standardized concordance score.

7 Step 7: standardize signals and fix their ordered menu

Code:
`bundlefuse/operators.py: zcols; bundlefuse/model.py; bundlefuse/basket_signals.py`

For every signal t and query j, define the catalogue mean $\mu_j^t = n^{-1} \sum_i r_{ij}^t$. Define ν as the standard-deviation correction: $\nu = 1$ on the general Torch path and $\nu = 0$ on the NumPy research basket path. Then

$$z_{ij}^t = \frac{r_{ij}^t - \mu_j^t}{\sqrt{\frac{1}{n-\nu} \sum_{i'} (r_{i'j}^t - \mu_j^t)^2} + 10^{-9}}. \tag{7.1}$$

This normalization is over catalogue items *within each query*, not over other queries, not just warm items, and not after seeds have been masked. Constant columns become zero. The Python types determine which standard deviation the shared `zcols` uses.

The auxiliary keys `pair`, `diff2_pair` and `rao` are standardized separately. In particular, z^{simp} is the normalization of $\boldsymbol{p} + \boldsymbol{\delta}$, not $z^{\mathrm{pair}} + z^{\mathrm{delta}}$. This matters for fusion.

The research menus below are ordered; coordinate selection visits keys in this order. Unavailable feature keys are removed on the general path, and family filtering is applied. Research basket menus are fixed and require their associated families.

POG / POG Dense

`cat, softcat, style, imgstyle, bridge, simp, diff2, pop, cooc, ppmi, csvd.` Description style may contribute to the bridge, but is not a separate selectable key here. Raw `content` is also excluded from this research menu.

Spotify / Spotify Sparse

`simp, diff2, pop, cooc, ppmi, csvd, cat, homo, audiostyle, bridge.`

Research retail

`simp, diff2, pop, cooc, hh_cooc, ppmi, csvd, cat, content, bridge.`

Research commerce

`simp, diff2, cooc, ppmi, csvd, cat, homo, tfidf, bridge, pretrain.`

The auxiliary keys are not themselves coordinate-selected weights. They are dependencies of the higher-order mixture. On the extended path the menu instead follows the full scorer's computed dictionary order, excluding auxiliary keys and applying the configured family filter. Appendix A specifies those additional signals.

8 Step 8: select the frozen fusion on validation queries

Code: `bundlefuse/fusion.py; bundlefuse/validation_cache.py; bundlefuse/operators.py: EmpiricalBayesRouter`

The selection routine needs a precise objective, score construction and search order. This section defines them in that dependency order. Validation labels affect metric-based choices; graph/content artifacts remain fixed during this search.

8.1 Define the ranking rule and validation objective first

Let $A^{\text{score}} \in \mathbb{R}^{n \times v}$ denote any candidate validation score matrix. For query j, let $\pi_{j1}, \pi_{j2}, \ldots$ be the items outside original S_j, ordered by decreasing A^{score}_{ij} and then increasing internal item ID for exact ties. Define $h_{jr} = \mathbf{1}[\pi_{jr} \in G_j]$ and $K_j = \min(K, n - |S_j|)$. For nonempty ground truth, the per-query metrics are

$$\text{Recall}_j @K = \frac{\sum_{r=1}^{K_j} h_{jr}}{|G_j|}, \tag{8.1}$$

$$\text{NDCG}_j @K = \frac{\sum_{r=1}^{K_j} h_{jr} / \log_2(r+1)}{\sum_{r=1}^{\min(K,|G_j|)} 1/\log_2(r+1)}. \tag{8.2}$$

The objective $\mathcal{J}$ is a macro mean over validation queries. By default it is mean NDCG@20. The accepted objective names also include `recallK`, `ndcgK`, `ndcgfull`, `mrr` and `nll`, where K in the first two names is a positive integer. Full NDCG uses the ranks of all distinct ground-truth items. MRR averages the reciprocal of the *first* relevant item's rank.

For `nll`, first re-standardize each candidate score column using NumPy's population standard deviation and epsilon 10^{-9}; call it $\widetilde{A}_{ij}$. Define $\ell_{ij} = \widetilde{A}_{ij}$ for non-seeds and $\ell_{ij} = -10^9$ for original seeds. The higher-is-better objective is the negative of

$$\text{NLL}(A^{\text{score}}) = \frac{1}{v} \sum_j \left[\log \sum_{i \in \mathcal{I}} \exp(\ell_{ij}) - \frac{1}{|G_j|} \sum_{i \in G_j} \ell_{ij} \right]. \tag{8.3}$$

The code uses a stable log-sum-exp calculation and float64 after this standardization. The seed mask is the finite value -10^9, rather than literal negative infinity.

8.2 Define observed evidence, prior and the final score

Define an alias map $a(t)$ that replaces `simp` by `pair`, replaces `diff2` by `diff2_pair`, and leaves every other signal key unchanged. For a weight dictionary w, define the pair-only and full-incidence standardized fusions

$$Y_{ij}^{(0)} = \sum_t w_t z_{ij}^{a(t)}, \qquad Y_{ij}^{(1)} = \sum_t w_t z_{ij}^{t}. \tag{8.4}$$

Given higher-order settings (α_h, γ_R), observed evidence is

$$Y_{ij} = (1-\alpha_h) Y_{ij}^{(0)} + \alpha_h Y_{ij}^{(1)} + \gamma_R z_{ij}^{\mathrm{rao}}. \tag{8.5}$$

The $\alpha_h = 0$ path uses only aliased pair signals. Increasing α_h mixes in *separately standardized* full signals. It does not simply add α_h times raw incidence surplus to a fixed normalized pair score. Concordance enters through its own standardized column and a single selected multiplier.

Before defining the prior, define the fallback key weight $\widetilde{w}_t = w_t$ if key t remains in the pruned dictionary, and $\widetilde{w}_t = 0.5$ otherwise. For prior-key list $\mathcal{P}$, define

$$P_{ij} = \sum_{t \in \mathcal{P}} \widetilde{w}_t z_{ij}^{t} + \gamma_R z_{ij}^{\mathrm{rao}} + \mathbf{1}[w_{\mathrm{pop}} > 0] w_{\mathrm{pop}} z_{ij}^{\mathrm{pop}}. \tag{8.6}$$

A missing popularity weight is interpreted as zero. The code includes the same concordance term in the observed score and the prior. Popularity is also explicitly added to the prior when selected, even though it was not one of the content/taxonomy prior candidates.

For nonnegative shrinkage κ and scalar prior scale s_0, define the prior mean $\mu_{0,ij} = s_0 P_{ij}$ and candidate-dependent prior fraction $u_i = \kappa/(f_i + \kappa)$ when $\kappa > 0$. The final score is

$$\widehat{\theta}_{ij} = \begin{cases} Y_{ij}, & \kappa \le 0, \\ (1-u_i) Y_{ij} + u_i \mu_{0,ij}, & \kappa > 0. \end{cases} \tag{8.7}$$

Thus a training-cold item ($f_i = 0$) receives the scaled prior when shrinkage is enabled; a high-exposure item receives more observed evidence. If selection disables shrinkage, even a cold item follows Y. Scores can be negative and are not probabilities.

8.3 Why this is called an empirical-Bayes router

For interpretation, define latent compatibility θ_{ij}, prior variance σ_0^2 and observation-noise coefficient σ_ϵ^2. The algebra of (8.7) matches the Gaussian assumptions

$$\theta_{ij} \mid \mu_{0,ij} \sim \mathcal{N}(\mu_{0,ij}, \sigma_0^2), \qquad Y_{ij} \mid \theta_{ij}, f_i \sim \mathcal{N}(\theta_{ij}, \sigma_\epsilon^2 / f_i), \quad f_i > 0, \tag{8.8}$$

with ratio $\kappa = \sigma_\epsilon^2/\sigma_0^2$. The implemented extension to $f_i = 0$ is the prior-only limit in (8.7). This is an interpretation of the scoring rule; the code neither tests Gaussianity nor returns a posterior uncertainty distribution. Moreover, the final ratio is selected from a validation grid, not forced to equal its moment estimate.

8.4 Stage A: coordinate search and backward pruning

Initialize every menu weight to zero and the incumbent objective to -10^{18}. For at most `fit_rounds` passes, visit the ordered menu. At coordinate t, try $w_t \in \{0, 0.25, 0.5, 1, 1.5\}$ while holding other coordinates fixed. Skip an all-zero candidate. Evaluate $\mathcal{J}(Y^{(0)})$, with concordance, incidence mixing and shrinkage disabled. Accept a candidate only if its score exceeds the incumbent by

2×10^{-4}. Stop after a pass with no accepted change. This is ordered greedy coordinate selection, not an exhaustive search over all weight combinations.

Remove zero weights. Try removing each listed optional key, retaining at least one positive-weight key; accept a removal if its objective is at least the current score minus 3×10^{-4}. The visited pruning keys are:

Path	Optional keys, in order
General	`bridge`, `contrast`, `softcat`, `homo`
Research retail	`bridge`, `contrast`, `cat`
Research commerce	`contrast`, `homo`

Absent keys, including unused `contrast` placeholders, are skipped. On the general path, prior keys are chosen next; on baskets they are chosen after the higher-order grid.

8.5 Stage B: choose prior keys and higher-order settings

The ordered general research prior candidates are `cat`, `softcat`, `style`, `imgstyle`, `bridge` for fashion and `cat`, `homo`, `audiostyle`, `bridge` for music. The extended general path uses `cat`, `softcat`, `homo`, `content`, `style`, `imgstyle`, `audiostyle`, `bridge`. Keep available candidates with positive selected weights. If none remain, use all available candidates in that order, allowing fallback weight 0.5 in (8.6).

Research retail uses positive-weight keys from `cat`, `content`, `bridge`, `contrast`; if empty, its fallback is `cat`, `bridge`. Research commerce uses positive-weight keys from `cat`, `homo`, `tfidf`, `bridge`, `pretrain`, `contrast`, `img`; its fallback is `bridge`, `tfidf`, `cat`. Actual menus exclude the unimplemented placeholders.

Evaluate the score in (8.5) with $\kappa = 0$, visiting α_h in $\{0, 0.5, 1\}$ as the outer grid and γ_R as the inner grid. The precise rules are:

Path	Concordance grid	Acceptance and incumbent
General	$0, 0.2, 0.35$	Start with $(0, 0)$ and its actual post-pruning score. Replace only for improvement $> 5 \times 10^{-5}$. If concordance is unavailable, use only $\gamma_R = 0$.
Retail	$0, 0.15, 0.25$	Start objective at -10^{18}. Replace when candidate score is $\geq$ incumbent minus 10^{-5}; a later near-tie can win.
Commerce	$0, 0.2, 0.35$	Start objective at -10^{18}. Replace only for improvement $> 10^{-5}$.

The retail rule permits small downward steps between successive accepted near-ties. It is not equivalent to taking the first global argmax. The basket function subsequently carries max(coordinate-search incumbent, accepted higher-order score) into its shrinkage comparison. Unlike the general selector, it does not simply reset that baseline to the final pruned higher-order score. These details are part of the current execution.

8.6 Stage C: freeze prior scale and estimate exposure moments

With selected weights, prior keys and higher-order settings, flatten Y and P over *all* validation queries and catalogue items. Let sd_ν be standard deviation with correction ν on this flattened

population. The scale is

$$s_0 = \begin{cases} \dfrac{\mathrm{sd}_1(Y) + 10^{-9}}{\mathrm{sd}_1(P) + 10^{-9}}, & \text{general streaming path,} \\ \dfrac{\mathrm{sd}_0(Y)}{\mathrm{sd}_0(P) + 10^{-9}}, & \text{NumPy research baskets.} \end{cases} \tag{8.9}$$

This one scalar is fitted once and frozen. It is not re-estimated on a test batch. The general selector computes global sums and squared sums in float64 across cached blocks; standard deviations are not averaged block by block.

For each warm item define its validation mean squared residual e_i, inverse exposure x_i, and their warm-item means $\overline{e}, \overline{x}$:

$$e_i = \frac{1}{v}\sum_{j=1}^{v}(Y_{ij} - s_0 P_{ij})^2, \qquad x_i = 1/f_i, \qquad f_i > 0. \tag{8.10}$$

These are residuals between two score constructions, not squared errors against binary relevance labels. The raw ordinary-least-squares estimates of slope b_0 and intercept a_0 are

$$b_0 = \frac{\sum_i (x_i - \overline{x})(e_i - \overline{e})}{\sum_i (x_i - \overline{x})^2}, \qquad a_0 = \overline{e} - b_0 \overline{x}, \tag{8.11}$$

where sums run over warm items. If fewer than two warm items exist or the denominator is $\leq 10^{-12}$, the components are marked unidentifiable and $\widehat{\kappa} = 0$. Otherwise define the admissible plug-in quantities

$$\widehat{\sigma}_0^2 = \max(a_0, 10^{-4}), \quad \widehat{\sigma}_\epsilon^2 = \max(b_0, 10^{-4}), \quad \widehat{\kappa} = \mathrm{clip}\left(\frac{\widehat{\sigma}_\epsilon^2}{\widehat{\sigma}_0^2}, 0.05, 50\right). \tag{8.12}$$

Raw estimates and truncation/identifiability flags are retained. These are clipped OLS moment estimates, not an iterative constrained-optimization fit. The general streaming path forms residuals and squares in the score dtype before float64 accumulation; NumPy basket estimation converts its input score arrays to float64 before forming residuals. One should not infer bit identity between those two numerical paths.

8.7 Stage D: choose the shrinkage setting

Use the following grids for κ, in the stated order:

Path	Candidate values
General	$0, 0.05, 0.5, 1, \widehat{\kappa}, 5$
Research retail	$0, 0.05, 0.25, \widehat{\kappa}, 1$
Research commerce	$0, 0.1, 0.5, \widehat{\kappa}, 2$

Unidentifiable variance components restrict every grid to the single value zero, even if positive constants were otherwise listed. The general selector tries the fitted scale s_0 and then scale 1, using the ratio grid inside each scale. It accepts objective improvements $> 5 \times 10^{-5}$. Basket selectors keep the fitted scale and accept improvements $> 10^{-4}$ relative to their carried incumbent. If scale 1 wins on the general path, it becomes the frozen s_0; the recorded moment estimates remain those fitted using the originally estimated scale. All start with shrinkage disabled. A grid may contain a repeated value; the code does not require deduplication.

The resulting frozen state is the ordered menu, pruned weight dictionary, prior-key list, (α_h, γ_R), (κ, s_0), variance diagnostics, graph/content artifacts, and dataset provenance. Test evaluation never changes these selected values.

8.8 How validation memory and replay work

The general path calls the independent all-signal scorer in blocks of `validation_chunk`, default 32. It writes each required standardized signal to a query-major float32 memory-mapped array with shape (v, n). The approximate cache payload for d_z retained signal keys is $4vnd_z$ bytes, excluding headers. All validation queries remain in the selection population; the block size limits resident scoring work, not the number of validation examples. Candidate scores are macro-averaged with query-count weights across blocks. Objective values are memoized by weights, higher-order settings, backoff and prior-key order. Independent candidates can share one pass through the cache.

The cache is temporary and is closed/deleted after selection, including on exceptions. It is not a fitted checkpoint. Research baskets instead compute their complete validation signal matrices in memory and call the NumPy selector; `validation_chunk` does not bound this basket validation allocation in the current code.

9 Step 9: save fitted state and restore it exactly

Code: `bundlefuse/model.py: save, load_model; evaluator.py; validate.py`

`fit` marks the object fitted only after selection succeeds. It does not save a file. A later `save(path)` makes a shallow model snapshot and a separate graph snapshot. For baskets it also copies the provider and rebinds that provider to the copied graph. The live fitted graph is not modified by checkpoint storage.

With `compressed=true`, the copied graph omits the derived attributes `C`, `C_bin`, `PPMI`, `W_pair_sq`, `W_csc`, and `P_rw`. It retains incidence matrices, exposure/size vectors, W, spectral features, selected weights/routing, content/context artifacts and provenance. The file is Python pickle with highest available protocol, despite its `.pt` extension. It is not a gzip archive, and compression here does not mean quantizing weights or changing retained numeric values. The optional fit-time content projection in Section 4 is a separate, score-changing approximation; current dataset JSONs disable that projection.

The snapshot is written to a temporary file in the destination directory, then atomically replaces the destination. Temporary partial-write files are cleaned up; the final checkpoint is retained. The ordinary evaluator's default destination is `checkpoints/<dataset>.pt`, so a later save to that same name replaces it. `validate.py` instead allocates a unique dataset/protocol/seed/UUID directory under `checkpoints/validation/`, or under the supplied checkpoint root.

On inference, `ensure_graph_matrices` reconstructs missing C from $B^\top B$ with its diagonal removed, and reconstructs M using exactly (3.4) when requested. These reconstructed matrices are retained on that loaded graph. It does not rebuild every omitted helper artifact. In particular, concordance converts retained W to CSC format if `W_csc` is missing, but keeps the conversion in a local variable: the current routine can repeat that conversion on later calls. Core prediction uses the factored L, so it does not need stored `P_rw` or `W_pair_sq`.

The custom unpickler remaps the earlier basket-provider module name to `bundlefuse.basket_signals.BasketSignals`. Loading an ordinary checkpoint does not refit SVD, clustering, weights, prior scale or shrinkage.

The fit CLI checks a validation-score probe before save, after save on the live model, and after reload; the comparison is NumPy array-value equality. The validation runner additionally compares every requested test score through its float32 unsigned-integer view, giving an actual bit-pattern equality check. Both write checkpoint paths/hashes and sizes to run reports. The

temporary signal cache and atomic-write staging files are the objects cleaned up automatically, not the validation runner's completed model files.

10 Step 10: execute a new prediction

Code: `bundlefuse/model.py: score_batch, score, predict; bundlefuse/selected_signals.py; bundlefuse/metrics.py`

10.1 Validate the query and choose scoring seeds

A fitted model accepts nonempty queries of distinct integer IDs in $[0, n)$; an explicit user list must align with the query count. Define the seed-cap setting $c_{\max} \geq 0$. The prepared scoring context is

$$S^\star = \begin{cases} S \text{ in its supplied order}, & c_{\max} = 0, \\ \text{first } \min(c_{\max}, |S|) \text{ IDs in sorted } S, & c_{\max} > 0. \end{cases} \tag{10.1}$$

This preparation is also used when computing validation signals. Original S remains available to metric/ranking code. Thus capped-away revealed items are neither scoring seeds nor recommendation candidates. Seed capping does not turn them into ground truth; the separate stress experiment below does change ground truth.

10.2 Determine dependencies of the frozen fusion

Let $\mathcal{W} = \{t : w_t \neq 0\}$ be the remaining positive-weight keys. Before computing scores, define the required set

$$\begin{aligned} \mathcal{D} = \{a(t) : t \in \mathcal{W}\} & \\ \cup \begin{cases} \mathcal{W}, & \alpha_h \neq 0, \\ \varnothing, & \alpha_h = 0 \end{cases} \cup \begin{cases} \{\text{rao}\}, & \gamma_R \neq 0, \\ \varnothing, & \gamma_R = 0 \end{cases} & \\ \cup \begin{cases} \{t \in \mathcal{P} : \widetilde{w}_t \neq 0\} \cup \{\text{pop} : w_{\text{pop}} > 0\}, & \kappa > 0, \\ \varnothing, & \kappa \leq 0. \end{cases} & \end{aligned} \tag{10.2}$$

This is `required_fusion_signals`. General `SelectedSignals` and basket `score_selected` compute exactly those keys and their operator dependencies. Indicators and overlap products are shared within a call; zero-weight/unneeded score families are skipped. Candidate items are not pruned: this still scores the full catalogue.

For example, if only `simp` has positive weight, $\alpha_h = 0$, $\gamma_R = 0$ and $\kappa = 0$, the required signal is only `pair`; incidence surplus, concordance, content and taxonomy are unnecessary. With $\alpha_h > 0$, both `pair` and `simp` are required. With $\kappa > 0$, any fallback-prior keys are required too.

10.3 Compute, standardize, fuse and rank

1. Restore only missing graph matrices required by $\mathcal{D}$.
2. Build indicators for $S^\star$, compute required raw scores from Section 6, and standardize them with (7.1).
3. Apply (8.5), then (8.6) and (8.7) when shrinkage is enabled. General scoring returns an $n \times q$ float32 NumPy array; basket fusion also returns NumPy scores.

4. For top-K prediction, copy each score column and assign $-\infty$ to *all* original revealed IDs S. Clamp output length to the number of available non-seed items.
5. Use partitioning to find the top-K boundary. Include all strictly higher items, fill boundary ties in ascending ID order, then sort the selected candidates by descending score and ascending ID.

`score_batch` returns unmasked full-catalogue scores; `score` extracts one column; `predict` adds seed exclusion and deterministic top-K selection. `reference=True` uses the all-signal reference scorer before the same fusion. The code's optimized and reference paths are intended to preserve operation order, while general floating-point batching can still select different low-level matrix kernels. Per-query standardization and frozen prior scale introduce no dependence on *other* query columns, but this is not a claim of bit equality across all BLAS kernels or batch shapes.

11 Step 11: run evaluation and bind it to the fitted data

Code: `inference.py; bundlefuse/cli_data.py; bundlefuse/evaluation.py; bundlefuse/provenance.py`

`inference.py` has three modes: direct/query-from-test prediction, test evaluation at a requested Recall/NDCG cutoff, and a fuller JSON report. No mode refits the model. The loaded object's model type and recorded dataset are checked against the requested CLI arguments. Direct `--seeds` prediction can run from a checkpoint alone; prediction with `--from-test`, evaluation and reporting reconstruct a dataset.

11.1 Recorded fingerprints and actual acceptance checks

The dataset specification records dataset name, item count, profile, split seed/mask, query limits and totals, complete/partial-fold flags, original query IDs, and hashes of training rows, validation/test pairs and available input files. Row hashes are SHA-256 of compact JSON serialization in the actual row order.

For checkpoint-bound reconstruction, omitted CLI options inherit the checkpoint's protocol, profile, split seed, mask and validation limit. A different dataset/profile/protocol, or a changed resplit seed/mask, is rejected. A legacy checkpoint missing the profile field is interpreted as extended. The loader then compares `n_items`, `train_sha256`, `val_pairs_sha256` and `input_sha256` with fitted metadata and rejects mismatches. An empty validation or test set is also rejected.

The test pair hash is recorded but is not in that four-key equality loop. Test query caps can be changed, for example to inspect a prefix through `--from-test`. Direct explicit-seed prediction does not read and re-hash the raw dataset files. This describes the implemented checks, rather than assuming that every stored field is always compared. Runtime metadata additionally hashes Python package sources and the three entry points, and records Python/platform/library versions and Torch thread count. Such hashes identify a run; ordinary inference does not reject every change in runtime source hash.

11.2 Macro test metrics and seed experiments

Metrics use the ranking and equations in Section 8.1. Evaluation scores query blocks, computes each block's macro metrics, multiplies by block query count, and divides the total by the complete number of evaluated queries. A small final block therefore has its correct weight. The standard full report includes cutoffs 10, 20 and 50.

Two different analyses appear in `report`:

Observed seed-count cohorts
Group existing test pairs by original $|S_j|$ and evaluate those unchanged pairs. The number of queries per cohort is reported; the model's seed cap can further restrict actual context.

Controlled seed stress
For requested count $c \in \{1, 2, 3\}$, let $S_j^{(c)}$ be the first $\min(c, |S_j|)$ sorted seeds and define

$$G_j^{(c)} = G_j \cup (S_j \setminus S_j^{(c)}). \tag{11.1}$$

Evaluate the new pair $(S_j^{(c)}, G_j^{(c)})$. Removed seeds become targets, preserving total bundle membership; the number of targets can increase. No fitting is repeated. This is a new completion task, not merely a timing shortcut for the original test pair.

`validate.py` is a separate integration runner: it fits, saves permanently, reloads, checks every requested test-score bit pattern, and records per-query metrics. It does not prove equality with an external published result unless an independent matched reference comparison is performed separately.

12 A small numerical walk-through

This example illustrates the equations; its frozen fusion settings are supplied for explanation, not claimed to be settings selected by the validation routine. Let $n = 5$, training bundles be $b_1 = \{0, 1, 2\}$ and $b_2 = \{0, 3\}$, and let original seeds be $S = S^\star = \{0, 1\}$. Item 4 is in the known catalogue but never appears in training. Use the default exponents and $\beta = 1.5$. Then

$$B = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}, \qquad \boldsymbol{f} = (2, 1, 1, 1, 0)^\top. \tag{12.1}$$

Define $a = 1/\sqrt{7}$ and $b = 1/\sqrt{6}$. The weighted overlaps and counts are

$$\boldsymbol{o} = \left(\frac{a+b}{2}, \frac{a}{\sqrt{3}}\right)^\top, \qquad \boldsymbol{k} = (2, 1)^\top. \tag{12.2}$$

The common pair score and surplus are therefore

$$\boldsymbol{p} = (o_1 + o_2, o_1, o_1, o_2, 0)^\top, \tag{12.3}$$

$$\boldsymbol{\delta} = (1.5o_1, 1.5o_1, 1.5o_1, 0, 0)^\top. \tag{12.4}$$

Only the first training bundle receives incidence activation, because only it overlaps both seeds. Suppose the frozen settings are $w_{\text{simp}} = 1$, $\alpha_h = 1$, $\gamma_R = 0$, prior keys $\{\text{content}\}$, $\kappa = 1$, and $s_0 = 1$. The prior key lacks a selected weight, so (8.6) assigns it weight 0.5. For illustration, supply raw content scores proportional to $(0, 1, 2, 3, 4)^\top$; their Torch-standardized column is approximately

$$\boldsymbol{z}^{\text{content}} = (-1.265, -0.632, 0, 0.632, 1.265)^\top. \tag{12.5}$$

The observations are $Y = \boldsymbol{z}^{\text{simp}}$, obtained by standardizing $\boldsymbol{p} + \boldsymbol{\delta}$ across all *five* catalogue items, including both seeds and item 4. The warm candidate weights are $u_2 = u_3 = 1/2$, while $u_4 = 1$. Consequently

$$\widehat{\theta}_2 = \tfrac{1}{2} z_2^{\text{simp}}, \qquad \widehat{\theta}_3 = \tfrac{1}{2} z_3^{\text{simp}} + \tfrac{1}{4} z_3^{\text{content}}, \qquad \widehat{\theta}_4 = \tfrac{1}{2} z_4^{\text{content}} \simeq 0.632. \tag{12.6}$$

The resulting candidate scores are approximately $\widehat{\theta}_2 = 0.288$, $\widehat{\theta}_3 = -0.273$, and $\widehat{\theta}_4 = 0.632$. For $K = 2$, the returned IDs are therefore $[4, 2]$. Original IDs 0 and 1 are excluded only at ranking. This shows how exposure controls the mixture: item 4 can receive content-prior evidence despite zero collaborative exposure, while item 2 receives the bundle-overlap surplus. The model does not infer a new graph edge or retrain a network to produce this cold-item score.

A Extended-profile behavior and additional signals

The extended profile uses the general model/scorer with a larger signal pool and different basket preprocessing. It is part of the current code and helps explain checkpoints fitted under earlier configurations; it is not interchangeable with the default research basket path.

A.1 Preprocessing differences

Extended retail reads the same capped/filterable basket pool but deduplicates by *household plus item set*, rather than item set alone. Missing product metadata share a common fallback category. TF–IDF/SVD uses the shared 8,000-feature helper; the model later normalizes its dense output. It builds training household histories and query users, and provides a weighted auxiliary household incidence to the general model.

Extended commerce uses joined title text only with the 8,000-feature helper, treats the category CSV field as a literal label rather than parsing category paths, and reads external pretraining interactions as user histories. It does not construct the research commerce session graph or two-hop sparse text bridge. Shared split/masking rules still apply. The profile settings remain independent: crossing a research loader with an extended scorer, or the reverse, follows the dispatch condition in Section 1; it does not recreate the paired research recipe automatically.

A.2 Auxiliary co-occurrence and group signals

Let $Z \in \mathbb{R}^{n_u \times n}$ be a supplied context-by-item incidence. The actual scorer uses $Z^\top Z\boldsymbol{x}$, or $Z^\top ZD\boldsymbol{x}$ when its weighted-seed flag is set. It does not remove the resulting diagonal. The basket loader's shared helper first normalizes context rows by $(\sum_i Z_{ui} + 1)^{-1/2}$. Extended retail uses this for household context with weighted seeds; extended commerce uses it for pretraining with unweighted seeds.

The optional `group_labels` API instead constructs item-by-group membership H_g, normalizes each group column by the square root of its catalogue size plus one, and sums $\omega_g \widetilde{H}_g \widetilde{H}_g^\top \boldsymbol{x}$ over supplied groupings with their provided weights ω_g. This supplies non-music `homo`. Without such groups, the general non-music fallback is raw category-count co-occurrence $A^\top A$, diagonal kept, scored using the seed labels. This fallback is not the commerce category-PMI formula.

A.3 Optional personalization

Let R_{iu} be the number of times item i occurs in the supplied history of user u, and let $\boldsymbol{h}_u$ be its binary item indicator. The general path supplies

$$r_i^{\text{userfreq}}(S, u) = R_{iu}, \qquad \boldsymbol{r}^{\text{usercooc}}(S, u) = C\boldsymbol{h}_u. \tag{A.1}$$

These have code keys `user_freq` and `user_cooc`. An unknown/omitted user gives zero columns. The scoring seed set does not define the history. Extended retail histories use training baskets;

extended commerce histories come from its supplied external pretraining interactions. The research basket provider ignores per-query user arguments and instead uses its shared training context graphs.

A.4 Extended menu order

For the general path, computed keys begin with `simp, diff2, pair, diff2_pair, pop`, then available `rao`, `cooc`, `ppmi`, and auxiliary keys in supplied dictionary order. They continue with `cat`, available `homo`, available `softcat`, feature keys in construction order, then available user keys. Feature construction begins with raw `content`, then styles in modality order, then `bridge`, then `csvd`; with no modalities it contains only `csvd`. The selectable menu filters out auxiliary mixture keys and disallowed families. Thus feature order and profile are relevant inputs to the ordered coordinate search, even when two paths contain similar mathematical score families.

B Baseline paths implemented beside BundleFuse

Code: `bundlefuse/baselines.py`

These baselines share the CLI and ranking/metric functions, but do not run BundleFuse's fusion search. Their graph fit constructs a common graph and retains incidence and frequency vectors. Define $A_B = B^\top B$ *with diagonal retained*, and define g_i as the zero-diagonal co-occurrence row mass plus 10^{-9} from (3.4). Define $D_1 = \mathrm{diag}((f_i+1)^{-1/2})$ and $G_1 = \mathrm{diag}(g_i^{-1})$. The implemented scores are

Name	Actual score computation
`popularity`	$\log(1+\boldsymbol{f})$, independent of seeds.
`cooccurrence`	$A_B\boldsymbol{x}$.
`itemknn`	$D_1A_BD_1\boldsymbol{x}$, with no explicit neighbor truncation.
`p3alpha`	Three iterations $\boldsymbol{w} \leftarrow G_1A_B\boldsymbol{w}$, starting at $\boldsymbol{x}$.
`rp3beta`	The preceding three-step output divided by $(f_i+1)^{0.5}$.
`contentknn`	Dot product of normalized concatenated content with the mean normalized seed feature; no explicit neighbor truncation.

The names are interface labels: the implemented `p3alpha` numerator retains a self-diagonal while its denominator uses off-diagonal degree, so it is not literally a row-stochastic walk on the matrix used in the numerator. No tunable alpha exponent is applied by that class. These exact baselines should not be silently identified with every published variant bearing a similar name. Their supplied features and data folds still follow the same chosen loader profile.

C Runtime cost and the timing entry point

Code: `scripts/benchmark_inference.py; bundlefuse/operators.py: build_indicator, score_rao_concordance`

Let d be the dense feature width, $k = |S^\star|$, and $\mathrm{nnz}(A)$ the stored nonzero count of sparse matrix A. Dense candidate dot products require approximately $O(nd)$ work per query after forming the seed mean. Sparse incidence/co-occurrence operations have costs driven by their nonzero structure. A dense score block requires $O(nq)$ memory per simultaneously materialized score family. Retaining item vectors does not turn this implementation into an approximate nearest-neighbor index: candidates are still scored over all n items.

Concordance forms an $n \times k$ support matrix; its dense-feature multiplication costs $O(ndk)$ and its direct support–kernel product costs $O(nk^2)$. A missing CSC copy of W introduces conversion/storage work on the order of $\mathrm{nnz}(W)+n$ per concordance call in the current implementation. The query indicator currently starts as an n-row LIL matrix before conversion to CSR, which also has allocation overhead. These are actual execution costs, not extra mathematical terms in the score.

The benchmark script hashes and loads retained checkpoints, reconstructs/fingerprints queries, performs one first request, warms up, and times the public `predict(k=20)` call on sampled distinct test queries in shuffled repetitions. Its default sample is 256 queries, three repetitions, up to 16 warm-up queries, and one thread for Torch and native thread pools. Dataset workers run sequentially in fresh processes. Timed online work includes input checks, scoring, fusion, exclusion and top-K ranking; it excludes checkpoint/query loading, warm-up, metric evaluation and consistency hashing. A separate batch-size-32 timing includes both `score_batch` and per-column top-K. It reports amortized batch throughput separately from single-request latency, and does not modify checkpoints. This explains what a timing measurement of this code represents without importing empirical performance numbers into the theory.

D Active pipeline versus helper methods

The call path above uses incidence surplus, `score_rao_concordance`, dense/sparse content bridges, validation selection and conjugate score mixing. Several other helpers exist in `operators.py` but are not called by the described `BundleFuse.fit` and ordinary prediction path: `propagate_rw`, `score_concordance_e2`, `score_simplicial_and_diff2`, `build_cold_warm_knn`, `fit_eb_moments`, `apply_tail_cold_calibration`, and `precision_backoff_weights`. In particular, this pipeline does not insert cold items into reserved top-K slots, perform an extra KNN cold-item transfer, or add the separate unweighted elementary-symmetric concordance helper. Their presence in the file is not evidence that those mechanisms execute. The theoretical interpretation must follow the caller, not every function name found in the module.

E Source map and reproduction

Each row below identifies the code responsible for a stage. Paths are relative to the repository root. The companion JSON manifest supplies full file hashes; no previous LaTeX notes or paper sections are inputs to the derivation.

Stage	Source / callable
Configuration and registry	`bundlefuse/config.py`: `make_config`; `bundlefuse/datasets/__init__.py`: `DatasetSpec`.
Provided folds and masks	`bundlefuse/splits.py`: `ProvidedSplits`, `mask`.
Data objects and full-ranking load	`bundlefuse/datasets/base.py`: `Dataset.fit_kwargs`, `load_bundlemllm`.
Research basket load	`bundlefuse/datasets/baskets.py`: `load_retail`, `load_commerce`.
Graph and collaborative features	`bundlefuse/operators.py`: `SimplicialGraphOperator.__init__`, `propagate`, `score_pairwise_and_diff2`.
Higher-order scores	Same class: `score_simplicial_incidence_only`, `score_rao_concordance`.
Content and ontology	`bundlefuse/operators.py`: `BilateralContentBridge`, `DualSpectrumOntology`; `bundlefuse/model.py`: `BundleFuse.fit`.
General full/selected scores	`bundlefuse/model.py`: `_score_all`; `bundlefuse/selected_signals.py`: `SelectedSignals.score`.

Stage	Source / callable
Research basket scores	`bundlefuse/basket_signals.py`: `BasketSignals`, `score_selected`.
Standardization	`bundlefuse/operators.py`: `zcols`.
Fusion and dependency selection	`bundlefuse/fusion.py`: `apply_fusion`, `apply_basket_fusion`, `required_fusion_signals`.
Validation selection	Same module: `fit_fusion`, `fit_basket_fusion`, `ValidationSignals`.
Validation storage	`bundlefuse/validation_cache.py`: `ValidationCache`.
Exposure moments and posterior	`bundlefuse/operators.py`: `EmpiricalBayesRouter`.
Public inference / checkpoint	`bundlefuse/model.py`: `score_batch`, `score`, `predict`, `save`, `load_model`.
Ranking and metrics	`bundlefuse/metrics.py`: `top_items`, `ranking_metrics`, `full_ndcg`, `mean_reciprocal_rank`.
Dataset binding and reports	`bundlefuse/cli_data.py`: `load_data`; `bundlefuse/evaluation.py`; `bundlefuse/provenance.py`.
Executables	`evaluator.py`, `inference.py`, `validate.py`, `scripts/benchmark_inference.py`.

To compile this standalone source from the repository root:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  -output-directory=output/pdf docs/pipeline_theory.tex
pdflatex -interaction=nonstopmode -halt-on-error \
  -output-directory=output/pdf docs/pipeline_theory.tex
```

Run twice to resolve the table of contents and cross-references. The generated PDF is `output/pdf/pipeline_theory.pdf`. The document and companion manifest describe the inspected working-tree snapshot; later code changes require checking the equations and updating the source hashes rather than merely changing a document date.